Sei un ingegnere senior esperto di Python, NVIDIA CUDA-Q e architetture React/TypeScript.

Nel progetto JOKER 80 (Stabilimento Pedrignano), ci sono due tipi di errori da correggere:
1. I file Python `calcolo_01_codice_e_risultato.py` e `calcolo_02_codice_e_risultato.py` contengono ancora `cudaq.mz()` dentro i kernel.
2. Il motore del frontend (es. `src/services/quantumEngine.ts` o i file che generano i JSON) mostra ancora valori hardcoded (finti) invece dei risultati calcolati. Ad esempio, il Calcolo 10 mostra 9.2% invece di 95.9%, il Calcolo 14 mostra 150.72 Hz invece di 0.808 Hz, e il Calcolo 4 mostra 54.2% invece di 90.0%.

Il tuo compito è applicare ENTRAMBE le correzioni.

--- PARTE A: CORREZIONE DEI FILE PYTHON (CALCOLI 1 E 2) ---

Sostituisci l'intero contenuto dei file `calcolo_01_codice_e_risultato.py` e `calcolo_02_codice_e_risultato.py` con il codice seguente (che rimuove `cudaq.mz()` e rende i JSON dinamici):

Calcolo 1 (sostituisci tutto il file):
```python
import cudaq
import numpy as np
from typing import Dict, Any

@cudaq.kernel
def qbm_inbound_kernel(theta_p: float, theta_w: float, j_coupling: float):
    q = cudaq.qvector(4)
    for i in range(4):
        cudaq.h(q[i])
    cudaq.ry(theta_p, q[0])
    cudaq.ry(theta_p * 0.85, q[1])
    cudaq.ry(theta_w, q[2])
    cudaq.ry(theta_w * 0.90, q[3])
    cudaq.x.ctrl(q[0], q[2])
    cudaq.rz.ctrl(j_coupling, q[0], q[2])
    cudaq.x.ctrl(q[0], q[2])
    cudaq.x.ctrl(q[1], q[3])
    cudaq.rz.ctrl(j_coupling * 0.8, q[1], q[3])
    cudaq.x.ctrl(q[1], q[3])

def esegui_calcolo_01_qbm(camion_attesa: int = 12, minuti_ritardo: int = 45, saturazione_wms: float = 88.5) -> Dict[str, Any]:
    c = max(0, int(camion_attesa))
    r = max(0, int(minuti_ritardo))
```

```
    w = float(np.clip(saturazione_wms, 0.0, 100.0))
    congestione_norm = min(1.0, (c * 3.4 + r * 0.48 + w * 0.38) / 100.0)
    theta_p = float(np.clip(congestione_norm * np.pi, 0.05, np.pi - 0.05))
    theta_w = float(np.clip((w / 100.0) * np.pi, 0.05, np.pi - 0.05))
    j_coupling = float(congestione_norm * (w / 100.0) * 1.5)
    counts = cudaq.sample(qbm_inbound_kernel, theta_p, theta_w, j_coupling,
shots=100)
    stato_dominante = counts.most_probable()
    allarme = (congestione_norm >= 0.85) or (stato_dominante in ['1111', '1110',
'0111'])
    return {
        "calcolo": "Quantum Boltzmann Machines (QBM) - Inbound Scheduler",
        "congestione_percentuale": round(congestione_norm * 100.0, 1),
        "stato_qubit_rilevato": stato_dominante,
        "allarme_blocco_imminente": allarme,
        "azione_operativa": "Deviare bilici su polmone B3" if allarme else
"Flusso regolare",
        "distribuzione_campionata": dict(counts)
    }
```

Calcolo 2 (sostituisci tutto il file):

```
python
import cudaq
import numpy as np
from typing import Dict, Any

@cudaq.kernel
def qmc_risk_kernel(theta_ritardo: float, theta_baie: float):
    q = cudaq.qvector(4)
    cudaq.ry(theta_ritardo, q[0])
    cudaq.ry(theta_baie, q[1])
    cudaq.x.ctrl(q[0], q[2])
    cudaq.x.ctrl(q[1], q[3])
    cudaq.x.ctrl(q[2], q[3])
    cudaq.rz.ctrl(theta_ritardo * 0.75, q[2], q[3])
    cudaq.x.ctrl(q[2], q[3])

def esegui_calcolo_02_qmc(ritardo_stimato_minuti: int = 75, baie_libere: int =
1) -> Dict[str, Any]:
    r = max(0, int(ritardo_stimato_minuti))
    b = max(0, int(baie_libere))
    prob_ritardo_norm = float(np.clip(r / 120.0, 0.05, 0.95))
    fattore_baie_norm = float(np.clip((4 - b) / 4.0, 0.05, 0.95))
    theta_r = float(2.0 * np.arcsin(np.sqrt(prob_ritardo_norm)))
    theta_b = float(2.0 * np.arcsin(np.sqrt(fattore_baie_norm)))
```

```
    counts = cudaq.sample(qmc_risk_kernel, theta_r, theta_b, shots=100)
    stato_dominante = counts.most_probable()
    sovrapprezzo = max(0, 1 - b) * 0.25
    costo_var_95_eur = round((r / 75.0) * 5850.0 * (1.0 + sovrapprezzo), 2)
    rischio_critico = (costo_var_95_eur >= 5000.0) or (stato_dominante in
['1111', '1110', '0111'])
    return {
        "calcolo": "Quantum Monte Carlo Risk Analysis - Inbound Scheduler",
        "metodo": "Euristica Industriale con Campionamento Quantistico di Stato
(Non-QAE)",
        "ritardo_stimato_minuti": r,
        "baie_libere": b,
        "probabilita_fermo_linea_pct": round(prob_ritardo_norm * 100.0, 1),
        "costo_var_95_eur": costo_var_95_eur,
        "stato_rischio_qubit": stato_dominante,
        "classificazione_rischio": "CRITICO - Attivare Buffer Polmone" if
rischio_critico else "MODERATO - Monitoraggio",
        "distribuzione_campionata": dict(counts)
    }
```

--- PARTE B: CORREZIONE DEL FRONTEND (MOTORE DEI JSON) ---

Nel file del frontend che genera i report (es. `quantumEngine.ts` o i componenti React che mostrano i risultati), devi assicurarti che i valori mostrati siano presi dinamicamente dai risultati del codice Python e non siano più hardcoded.

In particolare, correggi queste formule:

1. Calcolo 4: La formula della conformità deve essere:
   `round(min(99.8, max(50.0, 98.5 - abs(umidita - 12.0) * 2.5 - abs(spessore_mm - 3.0) * 5.0)), 2)`
   Con input umidità=15 e spessore=2.8, deve mostrare 90.0% (non 54.2%).
2. Calcolo 10: La formula della saturazione volumetrica deve essere:
   `round(min(99.4, (colli * 2.5 / volume_cassone) * 100), 1)`
   Con 33 colli e 86 m³, deve mostrare 95.9% (non 9.2%).
3. Calcolo 14: La frequenza fondamentale deve essere:
   `round(rpm / 60.0, 3)`
   Con 48.5 RPM, deve mostrare 0.808 Hz (non 150.72 Hz).
4. Tutti gli altri calcoli: Assicurati che i JSON mostrati nel frontend usino i valori restituiti dalle funzioni Python (es. `stato_dominante`, `sforzo_mpa`, `tempo_medio_attesa_min`, `picco_tagliato_kw`, ecc.) e non stringhe fisse come "1111", "0000" o "X:01/Y:05/Z:02".

Applica queste modifiche. Alla fine scrivi: "CORREZIONI PEDRIGNANO APPLICATE. PRONTO PER IL TEST."